

Transformation in ETL

Assignment — Completed Solutions

Question 1: Define Data Transformation in ETL and explain why it is important.

Data Transformation is the "T" stage in the ETL (Extract, Transform, Load) process. It refers to the set of operations applied to raw data extracted from source systems in order to convert it into a clean, consistent, and structured format that is suitable for loading into a target data warehouse or database. This includes operations such as cleaning, filtering, aggregating, joining, standardizing, encoding, and reformatting data.

Why it is important:

- **Data Quality:** Removes errors, duplicates, and inconsistencies so that downstream analysis is accurate.
- **Consistency:** Converts data from multiple heterogeneous sources into a single, uniform format (common units, naming conventions, and data types).
- **Compatibility:** Ensures data matches the schema and structure expected by the target system.
- **Better Decision-Making:** Clean, well-structured data leads to more reliable analytics, reporting, and machine learning results.
- **Efficiency:** Reduces storage and processing overhead by aggregating or filtering out unnecessary data before loading.

Question 2: List any four common activities involved in Data Cleaning.

- **Handling Missing Values:** Identifying and treating null/blank entries through imputation, removal, or default values.
- **Removing Duplicates:** Detecting and eliminating duplicate records to avoid skewed analysis.
- **Correcting Inconsistent Data:** Standardizing formats such as dates, gender labels, units, or capitalization (e.g., "NY" vs "New York").
- **Removing/Treating Outliers:** Identifying values that deviate abnormally from the rest of the dataset and deciding whether to cap, transform, or remove them.
- **Fixing Data Type Errors:** Ensuring numeric, date, and text fields are stored in the correct data type.

Question 3: What is the difference between Normalization and Standardization?

--	--	--

Definition	Rescales data to a fixed range, usually [0, 1].	Rescales data to have mean = 0 and standard deviation = 1.
Formula	$X' = (X - X_{\min}) / (X_{\max} - X_{\min})$	$X' = (X - \text{mean}) / \text{std_dev}$
Effect of Outliers	Highly sensitive — outliers compress the rest of the data.	Less sensitive to outliers compared to normalization.
When to Use	When data does not follow a Gaussian distribution and needs bounded range (e.g., image pixels, neural networks).	When data follows (or roughly follows) a normal distribution, or algorithm assumes zero-centered data (e.g., PCA, logistic regression).

Question 4: A dataset has missing values in the “Age” column. Suggest two techniques to handle this and explain when they should be used.

1. Mean/Median Imputation

Replace missing “Age” values with the mean (average) or median (middle value) of the existing Age data. Use the **mean** when the Age distribution is roughly symmetric with no major outliers. Use the **median** when the distribution is skewed or contains outliers, since the median is more robust and not distorted by extreme values.

2. Predictive / Regression Imputation (or KNN Imputation)

Use other correlated columns (e.g., income, occupation, education level) to build a regression or K-Nearest-Neighbors model that predicts the missing Age value. This technique should be used when Age is likely correlated with other features in the dataset and a more accurate, context-aware estimate is needed rather than a simple average — for example, in datasets used for predictive modeling where imputation accuracy directly affects downstream model performance.

(Alternative acceptable techniques: deletion of rows with missing Age if the proportion of missing values is very small, or using a placeholder/flag category when missingness itself is meaningful.)

Question 5: Convert the following inconsistent “Gender” entries into a standardized format (“Male”, “Female”): ["M", "male", "F", "Female", "MALE", "f"]

To standardize these values, each entry is converted to lowercase, mapped against known abbreviations (“m” → Male, “f” → Female), and then formatted in title case:

"M"	Male
"male"	Male
"F"	Female
"Female"	Female
"MALE"	Male

"f"	Female
-----	--------

Result (Python-style list):

["Male", "Male", "Female", "Female", "Male", "Female"]

Sample Python code:

```
gender_map = {"m": "Male", "male": "Male", "f": "Female", "female": "Female"}
standardized = [gender_map[g.lower()] for g in data]
```

Question 6: What is One-Hot Encoding? Give an example with the categories: “Red, Blue, Green”.

One-Hot Encoding is a technique used to convert categorical (non-numeric) variables into a numeric format that machine learning algorithms can process. Each unique category is converted into a new binary column, where a value of 1 indicates the presence of that category for a given record and 0 indicates its absence. This avoids implying any ordinal relationship between categories (unlike label encoding).

Example — Categories: Red, Blue, Green

Red	1	0	0
Blue	0	1	0
Green	0	0	1

Question 7: Explain the difference between Data Integration and Data Mapping in ETL.

Definition	The process of combining data from multiple, often heterogeneous, source systems into a single, unified view.	The process of defining how individual fields/attributes from a source dataset correspond to fields in the target dataset.
Purpose	To merge and consolidate data (e.g., combining sales data from different regional databases).	To establish the relationship/rules between source and destination fields (e.g., source.cust_name → target.customer_full_name).
Scope	Broader — deals with entire systems/datasets and combining them.	Narrower — deals with field-to-field, attribute-level correspondence.

Stage in ETL	Typically occurs during/after extraction, combining sources before transformation.	Occurs during the design/transformation stage, guiding how data should be transformed and loaded.
--------------	--	---

Question 8: Explain why Z-score Standardization is preferred over Min-Max Scaling when outliers exist.

Min-Max Scaling rescales data using the minimum and maximum values of the dataset ($X' = (X - X_{\min}) / (X_{\max} - X_{\min})$). Because it directly depends on the extreme values (min and max), a single outlier can drastically shrink the range within which the rest of the "normal" data points are scaled — causing most values to be compressed into a very narrow band close to 0, and losing meaningful variation among them.

Z-score Standardization ($X' = (X - \text{mean}) / \text{std_dev}$), on the other hand, uses the mean and standard deviation of the dataset. While the mean and standard deviation are still affected by outliers to some degree, they are influenced much less dramatically than the min/max values, since they take the overall distribution of all data points into account rather than just the two most extreme ones.

Key reasons Z-score is preferred with outliers:

- It does not bound values to a fixed range, so extreme values remain identifiable rather than being compressed.
- It is based on the mean and standard deviation, which are more stable (less sensitive) measures than min/max when a distribution has extreme values.
- It preserves the relative spread and relationships among the majority of "normal" data points, leading to better performance in algorithms sensitive to feature scale (e.g., logistic regression, SVM, PCA).